In [1]:
```
import pandas as pd
import numpy as np
import yfinance as yf
import matplotlib.pyplot as plt
from scipy import stats
```

In [3]:
```
start_date = "2016-01-01"
end_date = pd.Timestamp.today().strftime("%Y-%m-%d")

vix_df = yf.download("^VIX", start=start_date, end=end_date, progress=False)
spx_df = yf.download("^GSPC", start=start_date, end=end_date, progress=False)
```

```
C:\Users\malav\AppData\Local\Temp\ipykernel_48528\3788011013.py:4: FutureWarning: Y
F.download() has changed argument auto_adjust default to True
  vix_df = yf.download("^VIX", start=start_date, end=end_date, progress=False)
C:\Users\malav\AppData\Local\Temp\ipykernel_48528\3788011013.py:5: FutureWarning: Y
F.download() has changed argument auto_adjust default to True
  spx_df = yf.download("^GSPC", start=start_date, end=end_date, progress=False)
```

In [5]:
```
vix = vix_df.squeeze()
if isinstance(vix, pd.DataFrame):
    vix = vix.iloc[:, 0]
vix.name = "VIX"

spx_close = spx_df.squeeze()
if isinstance(spx_close, pd.DataFrame):
    spx_close = spx_close.iloc[:, 0]
spx_close.name = "SPX"

spx_ret = spx_close.pct_change().rename("SPX_ret")

data = pd.concat([vix, spx_ret], axis=1).dropna()
```

In [41]:
```
window = 5
threshold = 25

data["rolling_max"] = data["VIX"].rolling(window=2*window+1, center=True).max()
data["is_peak"] = (data["VIX"] == data["rolling_max"]) & (data["VIX"] >= threshold)
vix_peaks = data.index[data["is_peak"]]

print(f"Detected {len(vix_peaks)} VIX peaks above {threshold} from {start_date} to

peaks_table = data.loc[vix_peaks, ["VIX"]].sort_index(ascending=False)
print(peaks_table.head(15))
```

```
Detected 40 VIX peaks above 25 from 2016-01-01 to 2025-12-06:

                  VIX
Date
2025-11-20  26.420000
2025-10-16  25.309999
2025-04-08  52.330002
2025-03-10  27.860001
2024-12-18  27.620001
2024-08-05  38.570000
2023-03-13  26.520000
2022-12-12  25.000000
2022-11-09  26.090000
2022-10-11  33.630001
2022-09-27  32.599998
2022-09-13  27.270000
2022-06-13  34.020000
2022-05-09  34.750000
2022-04-26  33.520000
```

In [43]:

```
pre_days = 10
post_days = 10
event_matrix = []

for peak in peak_dates:
    window_data = data.loc[peak - pd.Timedelta(days=pre_days*2):
                           peak + pd.Timedelta(days=post_days*2)]
    if len(window_data) < pre_days + post_days + 1:
        continue
    window_data = window_data.iloc[-(pre_days + post_days + 1):]
    event_matrix.append(window_data["SPX_ret"].values)
event_matrix
```

Out[43]:
```
[array([ 0.00085327,  0.0078028 , -0.02496545,  0.01669591, -0.0215991 ,
         0.00053182, -0.01169386,  0.00519544,  0.0202837 , -0.01563798,
         0.01414434, -0.01086348,  0.00552858,  0.02476022, -0.00044324,
        -0.01874309,  0.00499204,  0.00152673, -0.01848125, -0.01415394,
        -0.00066362]),
 array([-1.87430908e-02,  4.99203893e-03,  1.52673256e-03, -1.84812463e-02,
        -1.41539357e-02, -6.63620353e-04, -1.88950277e-04, -1.23011618e-02,
         1.95180496e-02,  1.65166541e-02,  1.64804392e-02, -4.66571371e-03,
        -2.60329429e-05,  1.44541972e-02, -1.24543719e-02,  4.43978681e-03,
         1.13482754e-02, -1.87011448e-03, -8.12097634e-03,  2.38687918e-02,
         4.09430874e-03]),
 array([-0.00184071,  0.00313299, -0.00325797,  0.00580819,  0.00271206,
        -0.00165156,  0.01336408, -0.0359198 , -0.0180965 ,  0.01777017,
         0.01703267,  0.01356504,  0.0019486 , -0.00684748,  0.00535296,
        -0.00087158,  0.01525335,  0.00340862,  0.00700929,  0.00013477,
         0.0052592 ]),
 array([ 0.00060262,  0.0118412 , -0.00673197, -0.01089882,  0.00048899,
        -0.00064809, -0.02120855, -0.04097922,  0.01744092, -0.00500159,
        -0.03753642,  0.01493609,  0.01391458,  0.00261293,  0.01340246,
         0.01206911,  0.00037347, -0.00584139, -0.0054965 ,  0.00097355,
         0.01602838]),
 array([ 0.0214956 , -0.00025266, -0.01439192, -0.00036117, -0.00429953,
        -0.00551183, -0.03086443,  0.01862504, -0.01732726, -0.0065596 ,
         0.01566678,  0.01085133,  0.01055784, -0.00631669,  0.00560032,
         0.0062593 ,  0.02120886, -0.00250892, -0.00919901, -0.01970149,
        -0.00148192]),
 array([ 5.41950368e-03, -1.99930324e-04, -1.90867096e-02, -2.07734806e-02,
         8.64005861e-05, -1.53957145e-02, -1.57721063e-02, -2.05882283e-02,
        -2.71122545e-02,  4.95937428e-02,  8.56268078e-03, -1.24158251e-03,
         8.49248413e-03,  1.26849693e-03, -2.47567304e-02,  3.43357146e-02,
         7.01043485e-03,  9.69528502e-03,  4.09804545e-03,  4.51841878e-03,
        -1.46297842e-04]),
 array([-0.01705385, -0.0759697 ,  0.04939631, -0.04886845, -0.09511268,
         0.09287125, -0.11984055,  0.05995485, -0.05183076,  0.00470781,
        -0.04335951, -0.02929387,  0.09382774,  0.01153501,  0.06241416,
        -0.03368735,  0.03351601, -0.01601272, -0.04414243,  0.02282935,
        -0.01513713]),
 array([-0.01010466,  0.03057259, -0.02203044,  0.00581669,  0.02679359,
        -0.01788105, -0.0306748 ,  0.02293025, -0.00053942,  0.01391806,
         0.01471407, -0.00524238,  0.02658392, -0.00921245, -0.02805903,
         0.00424983,  0.00904056, -0.00697941,  0.01150463,  0.01687154,
         0.00013308]),
 array([-0.00053942,  0.01391806,  0.01471407, -0.00524238,  0.02658392,
        -0.00921245, -0.02805903,  0.00424983,  0.00904056, -0.00697941,
         0.01150463,  0.01687154,  0.00013308, -0.02050032, -0.01746272,
         0.01152482,  0.00392636,  0.03150119, -0.0104844 ,  0.0166511 ,
        -0.0077736 ]),
 array([ 0.00424983,  0.00904056, -0.00697941,  0.01150463,  0.01687154,
         0.00013308, -0.02050032, -0.01746272,  0.01152482,  0.00392636,
         0.03150119, -0.0104844 ,  0.0166511 , -0.0077736 ,  0.00235371,
         0.01228918,  0.0148273 , -0.00210791,  0.00481234,  0.00375123,
         0.00821083]),
 array([ 0.01364898, -0.0033687 ,  0.02621165,  0.01204158, -0.00779917,
        -0.00531309, -0.05894406,  0.01306084,  0.00831222,  0.0189624 ,
        -0.0036003 ,  0.00059422, -0.00564949,  0.0064951 ,  0.00430742,
        -0.02585515,  0.01095945, -0.02422691,  0.01468568,  0.01540988,
```

```
        0.00502213]),
array([ 0.00454125,  0.01588173, -0.01081853,  0.00782746, -0.00564361,
        0.0104662 , -0.00936254,  0.01340637,  0.00908205, -0.0034061 ,
        0.00284861,  0.00840694,  0.00167904,  0.00574708, -0.01231986,
       -0.0061904 ,  0.00739514, -0.00647339,  0.01242838, -0.00375025,
        0.0076705 ]),
array([ 0.00359635,  0.01019563,  0.00167304,  0.00673257, -0.00219496,
        0.007525  ,  0.01536591, -0.03512584, -0.00813303, -0.02775634,
        0.02014499, -0.0175848 ,  0.00053307,  0.01274183,  0.00521936,
       -0.00461895, -0.00841237, -0.01118258, -0.0115711 ,  0.01051795,
       -0.02372146]),
array([ 0.00521936, -0.00461895, -0.00841237, -0.01118258, -0.0115711 ,
        0.01051795, -0.02372146,  0.00298746,  0.01597672,  0.01611059,
       -0.00481266,  0.00825372,  0.00529291, -0.00957765,  0.01797271,
       -0.01397352,  0.01739675,  0.00801012,  0.00879353,  0.01641581,
       -0.00630692]),
array([ 0.01611059, -0.00481266,  0.00825372,  0.00529291, -0.00957765,
        0.01797271, -0.01397352,  0.01739675,  0.00801012,  0.00879353,
        0.01641581, -0.00630692, -0.00662314, -0.00152776,  0.00013492,
       -0.01632986,  0.00472733, -0.0021957 ,  0.00521893,  0.00344576,
       -0.01858952]),
array([ 0.00472733, -0.0021957 ,  0.00521893,  0.00344576, -0.01858952,
       -0.00302562, -0.03528788,  0.01194733, -0.01212955,  0.0123182 ,
        0.01779929,  0.02204705,  0.01946019, -0.00028772,  0.01169989,
       -0.00139979,  0.00765186, -0.00997853,  0.01361034,  0.01164805,
       -0.00479193]),
array([-0.0079492 , -0.00128511, -0.001265  , -0.00435926,  0.01292125,
        0.0017728 ,  0.00575765, -0.00351112, -0.00390628, -0.00207309,
        0.00074581,  0.00353659,  0.00872253, -0.00222738,  0.00134155,
        0.00643885, -0.01475483,  0.0070826 ,  0.00570984,  0.0148474 ,
        0.00549186]),
array([-0.00207309,  0.00074581,  0.00353659,  0.00872253, -0.00222738,
        0.00134155,  0.00643885, -0.01475483,  0.0070826 ,  0.00570984,
        0.0148474 ,  0.00549186, -0.00655475,  0.00041579,  0.00227564,
       -0.00375345, -0.00719003,  0.00813638,  0.01393563,  0.00031672,
       -0.00301061]),
array([-0.00719003,  0.00813638,  0.01393563,  0.00031672, -0.00301061,
        0.00361584, -0.00148883, -0.02567788,  0.00976063, -0.01931148,
        0.01605177,  0.01389822,  0.00100877,  0.01085332,  0.0038975 ,
        0.00739935, -0.00111352, -0.00034518,  0.00166246,  0.00471103,
       -0.00056927]),
array([-0.0003204 , -0.00441584, -0.0018549 , -0.00773284,  0.00125632,
        0.01135161, -0.02447881, -0.00475021,  0.02379076, -0.00808085,
       -0.01306622, -0.01341721,  0.01949597, -0.00535923,  0.01415464,
        0.00603031,  0.01039549,  0.0010154 ,  0.00649192, -0.00156968,
        0.00287938]),
array([ 0.00274809, -0.00667834,  0.00070346,  0.00816546,  0.00737334,
       -0.01043565, -0.00867391, -0.0214494 ,  0.01217314,  0.01491796,
       -0.00253005, -0.00851729, -0.00294341,  0.01055474, -0.00078388,
        0.00991129, -0.00212528,  0.00187682,  0.00116531,  0.00076888,
       -0.00049233]),
array([-0.00772258,  0.00227649, -0.00574664,  0.008474  , -0.00155114,
       -0.00911087, -0.01697731, -0.00081236,  0.00951961,  0.01213472,
        0.00146101, -0.00277638, -0.02036411,  0.00156918, -0.01190971,
        0.01149148, -0.0129859 ,  0.01052451,  0.00410279,  0.00829828,
       -0.00191372]),
```

```
array([-0.00319714,  0.00165713,  0.00229385, -0.02272482,  0.0132002 ,
       -0.01896131, -0.01181519,  0.01419444, -0.00844856,  0.01173087,
        0.02070708,  0.00308529, -0.00718108,  0.00954907, -0.00913617,
       -0.00747068,  0.01634846, -0.00874342, -0.01028768, -0.01138806,
        0.01777793]),
array([-0.01838795, -0.00968954, -0.01103738, -0.01891482,  0.00277174,
       -0.01217191, -0.00149664, -0.00538409,  0.02434765,  0.01888595,
        0.006863  ,  0.00942252, -0.02439108,  0.00515693, -0.00370171,
        0.00840121,  0.01451721, -0.01811573, -0.01896945, -0.0038406 ,
        0.01576672]),
array([ 0.02237268, -0.0024426 , -0.0154735 ,  0.01864269, -0.00525467,
       -0.00793404, -0.02951816, -0.00723375,  0.02569825, -0.00429181,
       -0.01296155, -0.007421  ,  0.02140857,  0.02238376,  0.01234782,
        0.01166229, -0.00043466,  0.01130411, -0.0122727 ,  0.01434391,
        0.00506617]),
array([-0.00020487,  0.0160576 , -0.00061848, -0.01475295, -0.02774005,
        0.00569794, -0.02814631,  0.00209805,  0.0247469 , -0.03628451,
        0.00567524,  0.00483713,  0.02986242, -0.03564971, -0.00567422,
       -0.0320371 ,  0.0024579 , -0.01646321, -0.00129596,  0.0238697 ,
       -0.0039464 ]),
array([-0.03628451,  0.00567524,  0.00483713,  0.02986242, -0.03564971,
       -0.00567422, -0.0320371 ,  0.0024579 , -0.01646321, -0.00129596,
        0.0238697 , -0.0039464 ,  0.02016963, -0.04039526, -0.00583378,
        0.00014614,  0.01855504, -0.0081208 ,  0.00945076,  0.01988326,
        0.02474226]),
array([ 0.01843102, -0.01634731,  0.0031374 ,  0.00952339, -0.01079395,
       -0.02379869, -0.0291103 , -0.03876843, -0.00377368,  0.0145925 ,
       -0.03251195,  0.00220087,  0.02447724, -0.00130157,  0.00953222,
        0.03056329, -0.00297307, -0.02014304, -0.00071175, -0.00875928,
        0.01055381]),
array([-0.0107365 , -0.00409506,  0.01834102,  0.00661072,  0.01527145,
        0.01058427, -0.04323661,  0.00338701, -0.01131774, -0.00718213,
        0.0068571 , -0.0112721 , -0.01711649, -0.00842758, -0.01723262,
       -0.01034053, -0.00212036,  0.01967214, -0.02112644, -0.0150667 ,
        0.02588389]),
array([ 0.0068571 , -0.0112721 , -0.01711649, -0.00842758, -0.01723262,
       -0.01034053, -0.00212036,  0.01967214, -0.02112644, -0.0150667 ,
        0.02588389,  0.0305837 , -0.00201795, -0.01024508, -0.02800362,
       -0.00749246, -0.00651918, -0.00329077,  0.02596564, -0.02366266,
        0.02648005]),
array([ 0.02588389,  0.0305837 , -0.00201795, -0.01024508, -0.02800362,
       -0.00749246, -0.00651918, -0.00329077,  0.02596564, -0.02366266,
        0.02648005,  0.01142757, -0.0066721 , -0.00795091,  0.02372482,
        0.01188198,  0.01626665, -0.00738772, -0.00608261,  0.02462638,
       -0.0074544 ]),
array([-0.0074544 , -0.00410123, -0.02500198, -0.01058599,  0.01361872,
        0.00961398,  0.00559789, -0.02077789,  0.05543448,  0.00924075,
       -0.00893578,  0.00871312, -0.00825205, -0.00308932,  0.00475858,
       -0.00388369,  0.01357999,  0.00591469, -0.00028304, -0.01544419,
       -0.00159187]),
array([-0.00086763, -0.00119466, -0.01789421, -0.01439919, -0.00186237,
        0.00752178, -0.00734958,  0.0142793 ,  0.00728966, -0.00605272,
       -0.02492168, -0.01113775, -0.00900752,  0.00103734,  0.01486799,
       -0.0144517 ,  0.0058681 , -0.00404962, -0.01202064,  0.01746133,
       -0.00254074]),
array([ 0.01614774,  0.00068717, -0.01532692,  0.00141479, -0.01845936,
```

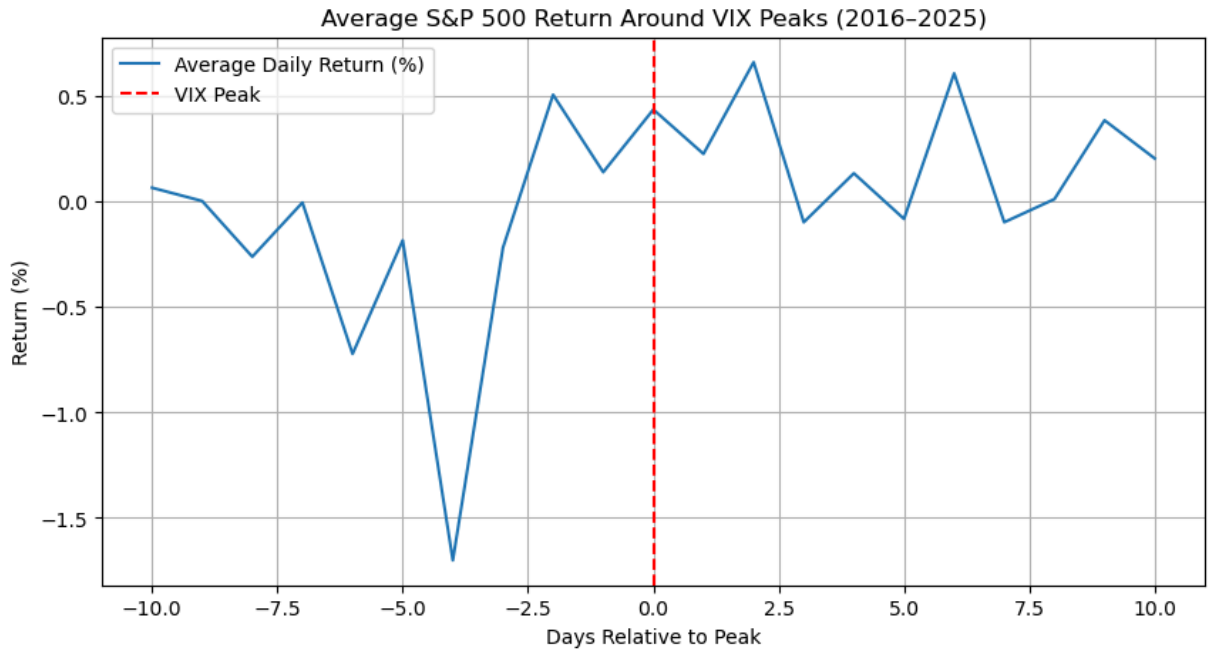
```
       -0.01447814, -0.00150976,  0.01647666, -0.00698088,  0.01756201,
       -0.01101946,  0.0089184 ,  0.01298219, -0.01646322,  0.00298453,
        0.00563981,  0.00164695, -0.00157384,  0.01423727,  0.00571527,
        0.01443654]),
array([ 1.10904687e-02,  8.13310129e-04, -4.96017188e-03,  1.57933987e-02,
       -1.36935030e-02, -1.83818609e-02, -2.99687984e-02,  1.03540859e-02,
       -7.73464786e-03,  2.30426115e-02,  4.67167685e-03,  4.30339777e-05,
        1.68475798e-02,  3.82372843e-03,  1.61332478e-02,  1.98977936e-03,
        9.72228474e-03, -1.98455540e-03,  4.23967683e-03, -8.93280555e-03,
        1.14833709e-02]),
array([ 2.49545384e-03, -6.14421393e-03, -2.96388331e-03,  8.16578605e-03,
       -5.41402253e-03, -2.64666391e-05,  3.79935433e-03, -3.86399497e-03,
       -2.94928464e-02, -8.65112325e-04,  1.08691238e-02,  7.28727347e-03,
        1.10427593e-02, -4.05658786e-04, -1.10557361e-02, -1.07019957e-02,
       -4.28480039e-03, -2.22388664e-03,  1.25960267e-02,  5.53804509e-03,
       -1.11039129e-02]),
array([ 0.01585414, -0.01759674, -0.01223483,  0.01115928, -0.01781901,
        0.00552062, -0.02697309, -0.00756787,  0.00488687, -0.01389098,
        0.02126587,  0.00641613, -0.01065351,  0.0107985 , -0.00218489,
        0.00082465,  0.017646  ,  0.00157433, -0.01115693, -0.00330698,
       -0.01973722]),
array([-0.01973722,  0.00553852,  0.00378124,  0.00672819, -0.04839563,
       -0.05974961, -0.00233147, -0.01570052,  0.09515388, -0.03460758,
        0.0180921 ,  0.00794471, -0.00172778, -0.02240837,  0.00132684,
       -0.02356749,  0.0251172 ,  0.01666114,  0.02025911,  0.00737313,
        0.00064071]),
array([ 5.82766364e-03, -2.75557041e-03, -2.71116732e-02,  1.55986718e-02,
       -1.56432666e-03,  4.02600116e-03, -6.29438710e-03,  5.27071555e-03,
        1.06722709e-02,  3.26964509e-05, -5.33753922e-03,  5.82739344e-03,
        7.90242259e-03,  1.22900509e-02,  2.28794386e-03, -4.35782552e-05,
       -9.90481244e-03,  2.61792171e-03,  1.72071273e-03, -1.17368301e-02,
        3.65355570e-03]),
array([-0.01117819,  0.00126184,  0.01540102,  0.00207535,  0.00062952,
       -0.01655685, -0.00050173, -0.00916227, -0.00825644,  0.00375384,
       -0.01556728,  0.00982304,  0.01546722,  0.0090617 ,  0.00690671,
        0.00535477, -0.00532333,  0.00245724,  0.00297979,  0.00108032,
        0.00193664])]
```

In [45]:

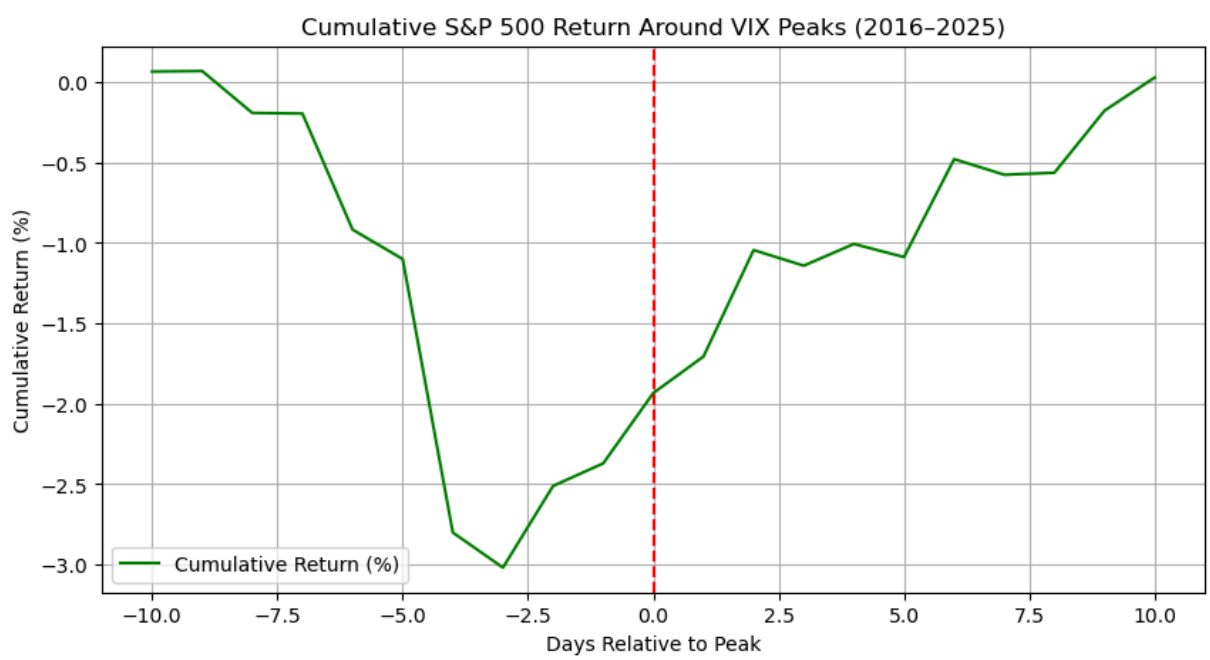
```
event_matrix = np.vstack(event_matrix)
AAR = event_matrix.mean(axis=0)
CAR = AAR.cumsum()
days = np.arange(-pre_days, post_days+1)
```

In [47]:

```
plt.figure(figsize=(10,5))
plt.plot(days, AAR*100, label="Average Daily Return (%)")
plt.axvline(0, color='red', linestyle='--', label='VIX Peak')
plt.title("Average S&P 500 Return Around VIX Peaks (2016–2025)")
plt.xlabel("Days Relative to Peak")
plt.ylabel("Return (%)")
plt.legend(); plt.grid(True); plt.show()
```



In [49]:
```
plt.figure(figsize=(10,5))
plt.plot(days, CAR*100, color='green', label="Cumulative Return (%)")
plt.axvline(0, color='red', linestyle='--')
plt.title("Cumulative S&P 500 Return Around VIX Peaks (2016–2025)")
plt.xlabel("Days Relative to Peak")
plt.ylabel("Cumulative Return (%)")
plt.legend(); plt.grid(True); plt.show()
```



In [17]:
```
pre_mean = AAR[days < 0].mean()
post_mean = AAR[days > 0].mean()
print(f"\nAvg daily return BEFORE peaks: {pre_mean*100:.3f}%")
print(f"Avg daily return AFTER peaks:  {post_mean*100:.3f}%")
```

```
Avg daily return BEFORE peaks: -0.237%
Avg daily return AFTER peaks:  0.196%
```

In [19]:
```
sp500 = pd.read_csv(
    "https://raw.githubusercontent.com/datasets/s-and-p-500-companies/master/data/c
)
```

In [51]:
```
def event_window(series, peaks, pre=10, post=10):
    mats = []
    for peak in peaks:
        window = series.loc[peak - pd.Timedelta(days=pre*2): peak + pd.Timedelta(da
        if len(window) >= pre + post + 1:
            mats.append(window.iloc[-(pre+post+1):].values)
    if not mats:
        return None, None, np.arange(-pre, post+1)
    mats = np.vstack(mats)
    AAR = mats.mean(axis=0)
    CAR = AAR.cumsum()
    return AAR, CAR, np.arange(-pre, post+1)
```

In [53]:
```
sp500["Symbol"] = sp500["Symbol"].str.replace(".", "-", regex=False)
```

In [55]:
```
# Drop known bad / delisted tickers that yfinance chokes on
bad_tickers = ["WBA"]
tickers = [t for t in sp500["Symbol"].unique().tolist() if t not in bad_tickers]
print(f"\nLoaded {len(tickers)} S&P500 tickers (after dropping bad ones)\n")

raw = yf.download(tickers, start=start_date, end=end_date, progress=False)

if raw.empty:
    raise RuntimeError("yf.download returned no data for S&P 500 tickers.")

# robust price extraction for MultiIndex vs normal DataFrame
if isinstance(raw.columns, pd.MultiIndex):
    lvl0 = raw.columns.get_level_values(0)
    if "Adj Close" in lvl0:
        prices = raw.xs("Adj Close", axis=1, level=0)
    elif "Close" in lvl0:
        prices = raw.xs("Close", axis=1, level=0)
    else:
        raise KeyError("Neither 'Adj Close' nor 'Close' found in MultiIndex columns
else:
    if "Adj Close" in raw.columns:
        prices = raw["Adj Close"]
    elif "Close" in raw.columns:
        prices = raw["Close"]
    else:
        raise KeyError("Neither 'Adj Close' nor 'Close' column found.")
```

```
Loaded 502 S&P500 tickers (after dropping bad ones)
```

```
C:\Users\malav\AppData\Local\Temp\ipykernel_48528\1616662275.py:6: FutureWarning: Y
F.download() has changed argument auto_adjust default to True
  raw = yf.download(tickers, start=start_date, end=end_date, progress=False)
```

In [57]:
```
# drop tickers with no data at all
prices = prices.dropna(axis=1, how="all")
```

```
# returns: don't forward-fill, just raw pct change and drop all-NaN rows
returns = prices.pct_change(fill_method=None).dropna(how="all")

print(f"Returns shape: {returns.shape}")
print(f"First 5 return columns: {list(returns.columns[:5])}")

print(f"Returns shape: {returns.shape}")
print(f"First 5 return columns: {list(returns.columns[:5])}")
```

```
Returns shape: (2496, 502)
First 5 return columns: ['A', 'AAPL', 'ABBV', 'ABNB', 'ABT']
Returns shape: (2496, 502)
First 5 return columns: ['A', 'AAPL', 'ABBV', 'ABNB', 'ABT']
```

In [59]:
```
#FUNDAMENTALS (SIZE + SECTOR)
# Compute realized volatility for each stock
volatility = returns.std().sort_values()

# Split by median volatility -> low-vol ≈ large-cap, high-vol ≈ small-cap
median_vol = volatility.median()
large_caps = volatility[volatility <= median_vol].index
small_caps = volatility[volatility > median_vol].index

print(f"Large-cap (low-vol) count: {len(large_caps)}")
print(f"Small-cap (high-vol) count: {len(small_caps)}")
```

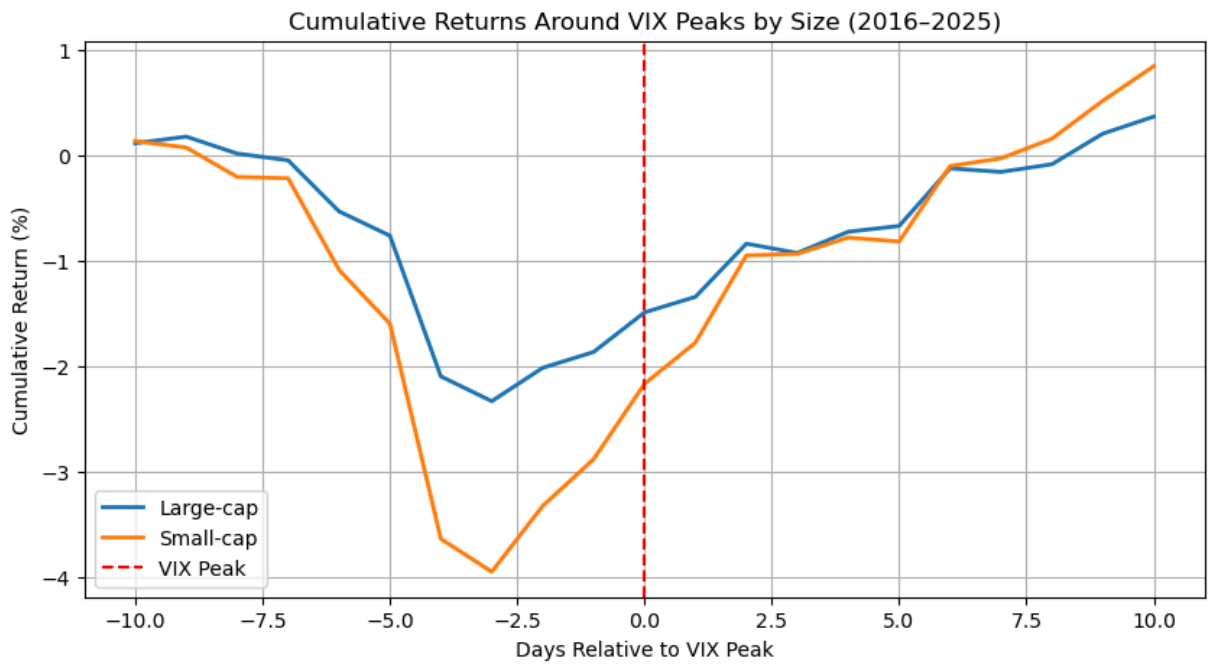
```
Large-cap (low-vol) count: 251
Small-cap (high-vol) count: 251
```

In [61]:
```
#SECTOR INFO
sector_info = {}
for t in returns.columns:
    try:
        tkr = yf.Ticker(t)
        sector_info[t] = tkr.info.get("sector", "Unknown")
    except:
        sector_info[t] = "Unknown"

meta = pd.DataFrame({"sector": sector_info})
if len(large_caps) == 0 or len(small_caps) == 0:
    raise RuntimeError("Could not form size portfolios — not enough usable tickers
```

In [62]:
```
port_large = returns[large_caps].mean(axis=1)
port_small = returns[small_caps].mean(axis=1)

sector_ports = {}
for sector in meta["sector"].dropna().unique():
    tickers_in_sector = meta[meta["sector"] == sector].index
    tickers_in_sector = [t for t in tickers_in_sector if t in returns.columns]
    if len(tickers_in_sector) > 5:
        sector_ports[sector] = returns[tickers_in_sector].mean(axis=1)
```

In [63]:
```
#  EVENT STUDY: SIZE
AAR_large, CAR_large, days = event_window(port_large, vix_peaks)
AAR_small, CAR_small, _ = event_window(port_small, vix_peaks)
```

```
plt.figure(figsize=(10,5))
plt.plot(days, CAR_large*100, label="Large-cap", lw=2)
plt.plot(days, CAR_small*100, label="Small-cap", lw=2)
plt.axvline(0, color="red", linestyle="--", label="VIX Peak")
plt.title("Cumulative Returns Around VIX Peaks by Size (2016–2025)")
plt.xlabel("Days Relative to VIX Peak")
plt.ylabel("Cumulative Return (%)")
plt.legend(); plt.grid(True); plt.show()
```

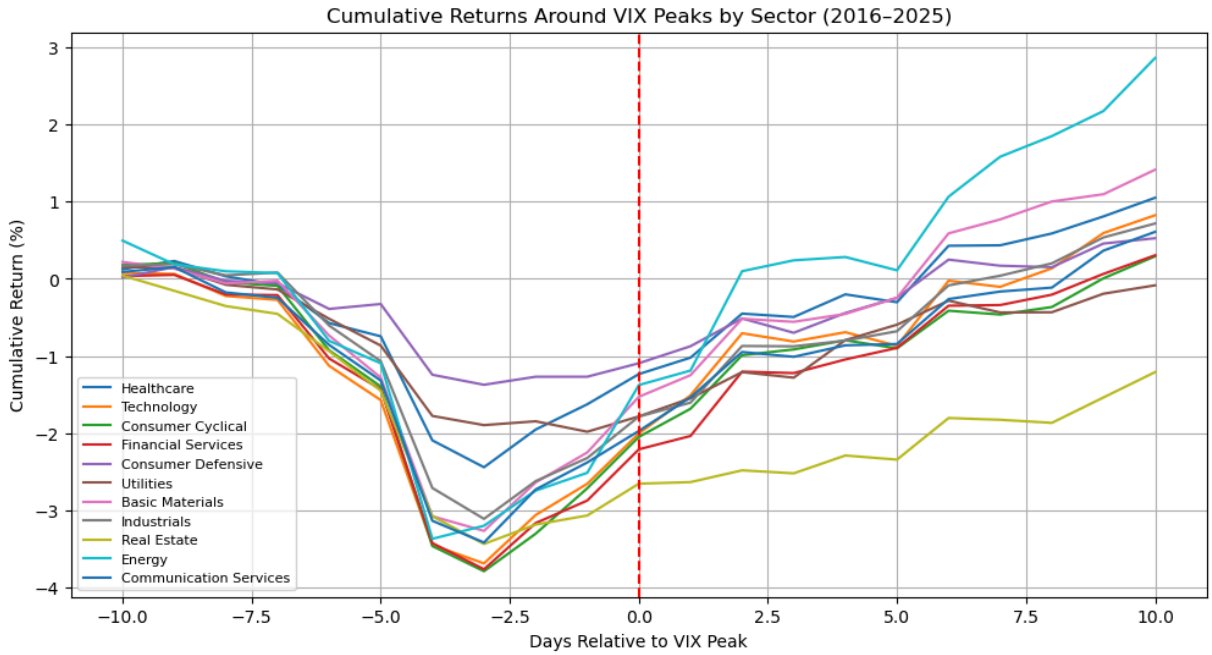


In [67]:
```
pre_large = AAR_large[days < 0].mean(); post_large = AAR_large[days > 0].mean()
pre_small = AAR_small[days < 0].mean(); post_small = AAR_small[days > 0].mean()

print(f"\nLarge caps: pre {pre_large*100:.3f}%, post {post_large*100:.3f}%")
print(f"Small caps: pre {pre_small*100:.3f}%, post {post_small*100:.3f}%")

if post_small > post_large:
    print("→ Small-caps rebound more strongly after VIX peaks (supports overreactio
else:
    print("→ No clear size asymmetry in rebound.")
```

```
Large caps: pre -0.187%, post 0.186%
Small caps: pre -0.288%, post 0.301%
→ Small-caps rebound more strongly after VIX peaks (supports overreaction hypothesi
s).
```

In [69]:
```
#  EVENT STUDY: SECTORS
plt.figure(figsize=(12,6))
for sector, port in sector_ports.items():
    _, CAR_sector, _ = event_window(port, vix_peaks)
    if CAR_sector is not None:
        plt.plot(days, CAR_sector*100, lw=1.5, label=sector)
plt.axvline(0, color='red', linestyle='--')
plt.title("Cumulative Returns Around VIX Peaks by Sector (2016–2025)")
plt.xlabel("Days Relative to VIX Peak")
plt.ylabel("Cumulative Return (%)")
plt.legend(fontsize=8)
```

```
plt.grid(True)
plt.show()
```



In [71]:
```
#  SECTOR SUMMARY TABLE
sector_summary = []
for sector, port in sector_ports.items():
    AAR_sector, _, _ = event_window(port, vix_peaks)
    if AAR_sector is not None:
        pre = AAR_sector[days < 0].mean()
        post = AAR_sector[days > 0].mean()
        sector_summary.append((sector, pre, post, post - pre))
sector_summary = pd.DataFrame(sector_summary, columns=["Sector", "Pre", "Post", "Po
print("\nAverage sector reactions (daily returns):")
print(sector_summary.sort_values("PostMinusPre", ascending=False).head(10))
```

```
Average sector reactions (daily returns):
                    Sector       Pre      Post  PostMinusPre
9                   Energy -0.002515  0.004242      0.006757
1               Technology -0.002658  0.002830      0.005488
3       Financial Services -0.002876  0.002516      0.005392
6          Basic Materials -0.002252  0.002945      0.005197
2        Consumer Cyclical -0.002717  0.002342      0.005059
10  Communication Services -0.002381  0.002582      0.004964
7              Industrials -0.002324  0.002504      0.004828
8              Real Estate -0.003069  0.001447      0.004516
0               Healthcare -0.001627  0.002287      0.003913
5                Utilities -0.001983  0.001700      0.003683
```

In [ ]: